

Rapport Final - Code Reader

Table des matières

- [Introduction](#)
- [Explication du projet](#)
- [Explication des fonctionnalités](#)
- [Conclusion](#)
- [Médiagraphie](#)

Introduction

Ce projet est une application WPF en C# qui sert à faire une communication directe avec Ollama et permet de télécharger et de communiquer avec une sélection de modèle de langage, dans des buts d'aider à la programmation et au débbugging. L'application sert d'interface visuelle capable de faire une communication simple avec un LLM qui fonctionne localement via Ollama pour lui poser des questions ou envoyer des morceaux de code afin d'avoir un avis ou une aide spécifique. Elle permet aussi d'envoyer directement des fichiers pour aider à l'analyse, et même d'un dossier complet, assumant que la taille de tout les fichiers ne dépassent pas la limite imposé par des modèles, ce qui aurait causé un manque de données dû à un trop petit contexte. L'application fonctionne complètement hors-ligne, quoique l'installation des modèles et de Ollama est nécessaire au fonctionnement.

Explication du projet

Le projet part d'une application WPF avec une architecture MVVM. Le coeur de l'application tourne autour de OllamaSharp, une librairie conçue pour faire la communication avec Ollama via du code C#, utilisant le REST API de ce dernier. Elle fourni la classe OllamaApiClient, qui est au centre de toutes les communications avec les modèles. Le NuGet OllamaSharp.ModelContextProtocol est aussi utilisé afin de raffiner ce que l'agent peut faire lors d'une réponse. Les MCPs inclus sont Context7, SequentialThinking et GoogleSearch.

Context7 permet aux LLMs d'aller chercher directement la documentation d'un langage afin d'être plus à jour dans leur analyse. Elle demande une clé d'API pour fonctionner, mais n'est pas complètement nécessaire puisqu'elle n'est qu'un outil de recherche. Une fenêtre de configuration à été ajouter afin de permettre l'ajout de sa propre clé d'API (puisque'elle n'est naturellement pas fournie) si désiré.

SequentialThinking donne aux agents une certaine capacité de réflexion afin de mieux organiser leur réponse et avoir de meilleurs résultats. Son utilisation est très visible lorsque le modèle donne une liste de point, où elle va expliquer chaque point plus en détail et de façon plus logique comparé à si il n'était pas présent. C'est un petit plus côté logique.

GoogleSearch agit comme un simple moteur de recherche afin de trouver des problèmes plus obscures. Elle n'utilise pas l'API de Google directement, mais à la place se base sur Playwright pour passer au travers des mécanismes anti-IA pour pouvoir faire des recherches directement.

Le tout combiné à un prompt initial pour mené le LLM en fonction à être spécifiquement un assistant pour la programmation.

Explication des fonctionnalités

Lors de l'ouverture du programme, il y aura une fenêtre qui dit "Hors Ligne". Après une sélection d'un modèle parmi la liste (qui est *hard-coded* mais facilement modifiable), et si l'usagé désire que le modèle aie un contexte plus petit ou plus grand via la case à cocher (nécessaire pour l'analyse de dossier), l'ouverture du modèle commencera après avoir cliqué sur Démarrer. Si le modèle n'est pas présent sur la machine, le programme le détectera et demandera si l'utilisateur veut l'installer. Si oui, l'installation commencera en arrière-plan et le modèle agira une fois prêt.

3 options sont offert, l'envoi d'un message, l'envoi d'un fichier et l'envoi d'un dossier. L'envoi de dossier est bloqué de base puisque la taille possible d'un dossier de projet pourrait dépassé la limite du contexte de base, et donc requiert que la case "Contexte élevé?" soit cochée, sinon la limite de token serait trop facile à dépasser. Lors d'un envoi, l'utilisateur ne peut pas faire un autre envoi avant que l'IA ne répondent. Dans le cas où une réponse n'a pas été envoyé après 3 minutes, elle est annulée et le contrôle est redonné à l'utilisateur. L'envoi de dossier ignore la majorité des extensions commune et dossier non-important pour l'analyse afin d'aider à avoir plus de budget de token pour une meilleure réponse. Le contenu du message de l'utilisateur est aussi envoyé automatiquement lors de la sélection d'un fichier ou dossier.

Lorsque vous avez fini votre conversation, vous pouvez arrêter manuellement le modèle afin d'en choisir un autre ou fermer l'application. Le modèle sera fermé lors de la fermeture afin de ne pas avoir une session ouverte en arrière-plan, ce qui poserais problème si le modèle choisi est très demandant.

Conclusion

Il manque définitivement quelques fonctions de qualité de vie à ajouter à l'application présentement, mais il s'agissait d'une assez bonne expérience d'apprentissage sur l'hébergement de petits LLMs sur une machine d'utilisateur, un bon défi de faire une interface de communication en WPF, utilisant beaucoup les fonctions asynchrones pour le bon fonctionnement de l'application, et une bonne leçon sur à quel point les algorithmes des LLMs sont customisable. Ce que j'aurais voulu ajouter encore est un affichage de Markdown et de ne pas avoir commencer avec une interface qui ne permet pas de surligner le texte (ce qui est étonnamment un gros point négatif de l'application présentement), mais ce sont des points à rajouter plus tard si je continue à travailler sur ce projet.

Médiagraphie

Introduction - Ollama. (2025). Ollama.com; Ollama. <https://docs.ollama.com/api/introduction>

Introduction - Model Context Protocol. (2025). Modelcontextprotocol.io; Model Context Protocol. <https://modelcontextprotocol.io/docs/getting-started/intro>

Context7 - Up-to-date documentation for LLMs and AI code editors. (2025). Context7.com. <https://context7.com/>

modelcontextprotocol. (2025). *servers/src/sequentialthinking at main · modelcontextprotocol/servers*. GitHub. <https://github.com/modelcontextprotocol/servers/tree/main/src/sequentialthinking>

modelcontextprotocol-servers. (2025). *GitHub - modelcontextprotocol-servers/google-search-mcp*. GitHub. <https://github.com/modelcontextprotocol-servers/google-search-mcp>

Microsoft. (2025). *Fast and reliable end-to-end testing for modern web apps | Playwright*. Playwright.dev. <https://playwright.dev/>

Getting started | OllamaSharp. (2026). Github.io. <https://awaescher.github.io/OllamaSharp/docs/getting-started.html>